

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of MONDAY 2 SEPTEMBER 2024 – Theory Part

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. A. MORZENTI

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): syntax and semantics of languages, divided in four sections:
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): compiler design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- For the theory part to be valid, the candidate must achieve a grade of at least 10/20 on each of the four sections.
- Correctly answering all the questions that are not marked optional, allows the candidate to achieve a high grade (although not the full mark).
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30.
- The final grade is the weighted average of the theory part (80 %) and of the lab part (20 %), and must be $\geq 18/30$.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

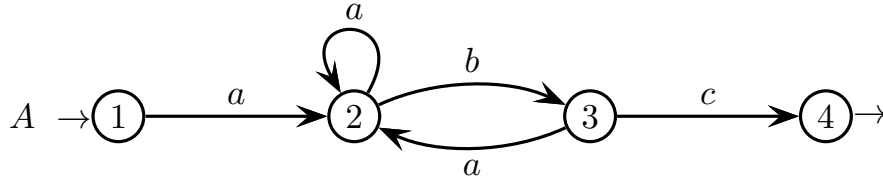
1 Regular Expressions and Finite Automata 20%

1. Consider the two regular expressions R_1 and R_2 below, over the four-letter alphabet $\{a, b, c, d\}$:

$$R_1 = ((a^+ b)^+ c)^+ d$$

$$R_2 = a^* b^* c^* d^*$$

and the automaton A below, over the three-letter alphabet $\{a, b, c\}$:



Answer the following questions:

- (a) Consider the regular expression R_1 above. Write one string of length 5 and two strings of length 6 generated by R_1 .
- (b) Write a derivation from the regular expression R_1 above, for the sample valid string below (of length 7):

$a a b a b c d$

- (c) Apply the Berry-Sethi (BS) method to the regular expression R_1 above, to obtain an equivalent deterministic automaton A_1 .
- (d) Say if the automaton A_1 obtained through the BS method at point (c) is minimal, and explain your answer. Minimize automaton A_1 if it is not minimal.
- (e) Apply the Brzozowski-McCluskey (BMC – node elimination) method to the automaton A above (**not to** A_1), to obtain an equivalent regular expression R .
- (f) (optional) Consider the regular expressions R_1 and R_2 above and their intersection language $L_\cap$, below:

$$L_\cap = L(R_1) \cap L(R_2) = L(((a^+ b)^+ c)^+ d) \cap L(a^* b^* c^* d^*)$$

Write a regular expression $R_\cap$ that generates language $L_\cap$, without using the intersection operator “ $\cap$ ” of sets. Briefly explain your answer.

2 Free Grammars and Pushdown Automata 20%

1. Consider the two languages L_1 and L_2 defined below, over the three-letter alphabet $\Sigma = \{a, b, c\}$:

$$L_1 = \{ a^i c b^{j+i} \mid 1 \leq i \leq 3 \wedge j \geq 0 \}$$

$$L_2 = \{ a^i c^k b^{j+i} \mid 1 \leq i \leq 3 \wedge j \geq 0 \wedge k \leq j \}$$

Answer the following questions:

- (a) Write a non-ambiguous BNF grammar G_1 that generates language L_1 .
(b) Draw the syntax tree of the sample valid string below, with grammar G_1 :

$a a c b b b b$

- (c) Is language L_1 regular or context-free? Justify your answer.
(d) Write a BNF grammar G_2 that generates language L_2 (possibly modify grammar G_1). It does not matter that grammar G_2 is ambiguous or not.
(e) (optional) Consider the intersection language $L_3 = L_1 \cap L_2$. Write down language L_3 as a string pattern constrained by a characteristic predicate, that is:

$$L_3 = \{ \text{string model(s)} \mid \text{characteristic predicate} \}$$

Does the sample string below belong to language L_3 ?

$a c b$

Is language L_3 regular or context-free? Justify your answers.

2. Consider the fragment of a language that describes multivariate (with one or more different variables) polynomials. This fragment features the structure as shown in the two sample polynomials below:

$$3 + 5 x^2 + 2 x^3 y$$

$$2 x + 3 y^2 (8 + 4 x z)^2$$

The language has the following characteristics:

- a polynomial is a sum “+” of monomials
- a monomial is the product (the multiplication sign is constantly understood) of one coefficient (a positive integer), mandatory, and of zero, one or more variables (identifiers), each of them possibly with a positive integer exponent, marked by the operator “^”; in place of any variable, there may be a subexpression consisting of a polynomial enclosed in parentheses
- addition is left-associative and multiplication is right-associative
- the precedence order from high to low is first exponentiation, then multiplication and finally addition
- for simplicity, positive integers and variables are schematized by the terminals `num` and `var`, respectively; neither terminal has to be further expanded

Answer the following questions:

- Write an EBNF non-ambiguous grammar that generates the language described before (the axiom must generate one polynomial).
- Draw the syntax tree of the sample polynomial shown below:

$$5 x + 2 x y^3$$

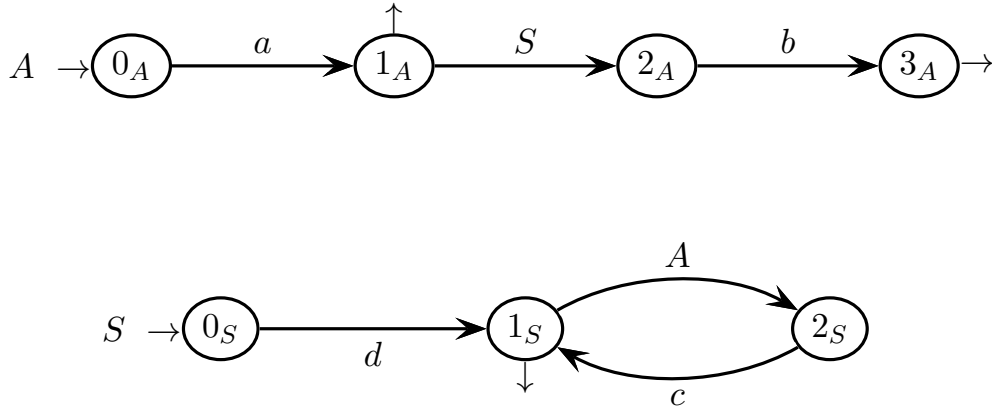
- (optional) Suppose the subtraction “−” of monomials is also possible, like:

$$3 + 5 x^2 - 2 x y^3$$

Subtraction is not associative, i.e., the grammar and the tree should not enforce either type (left or right) of associativity. Subtraction takes precedence over addition and gives precedence to multiplication. Accordingly modify the previous grammar, still non-ambiguous (you may show only the modified or new rules).

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G below, represented by a machine net with the two-symbol nonterminal alphabet $V = \{S, A\}$ (axiom S) and the four-letter terminal alphabet $\Sigma = \{a, b, c, d\}$:



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw the syntax trees for the two sample valid strings below:

$$d a d b c \in L(G) \qquad d a c a d b c \in L(G)$$

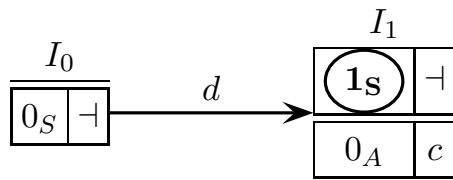
- (b) Draw the complete pilot of grammar G and argue that grammar G is ELR(1).
- (c) Simulate the ELR(1) parsing process of the sample valid string below:

$$d a d b c \in L(G)$$

- (d) Write all the guide sets on the call arcs and exit arrows of the net of grammar G , and argue that grammar G is ELL(1).
- (e) (optional) Write the recursive-descent syntactic procedure for nonterminal S .

syntax trees – question (a)

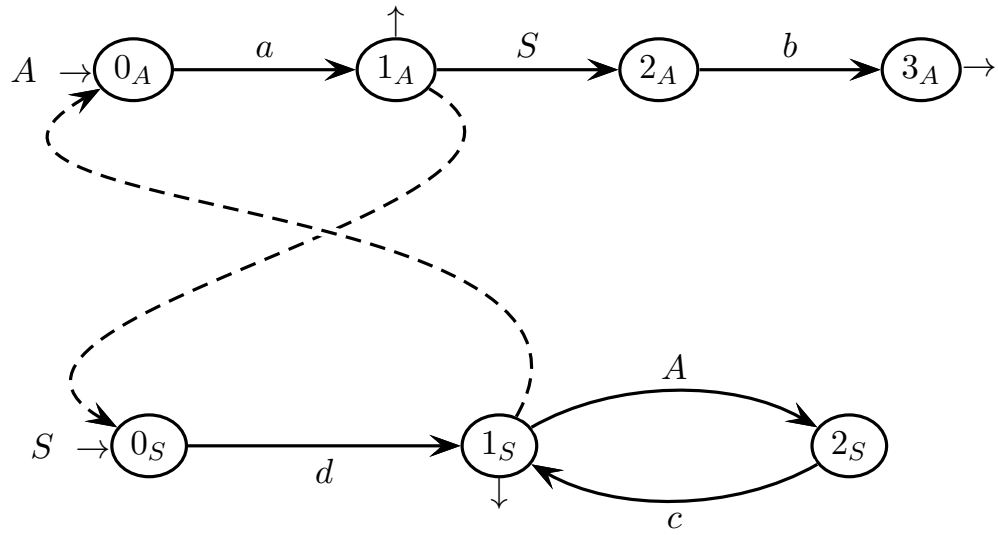
pilot to be completed and ELR analysis – question (b)



ELR(1) parsing process (nom. of rows not significant) – question (c)

[illegible]

guide sets and ELL analysis – question (d)



recursive-descent procedure – question (e)

4 Language Translation and Semantic Analysis 20%

1. Consider the following source grammar G , over the two-symbol nonterminal alphabet $V = \{S, L\}$ (axiom S) and the three-letter terminal alphabet $\Sigma = \{a, "[", "]" \}$. Grammar G generates nested lists delimited by square brackets:

$$G \left\{ \begin{array}{l} S \rightarrow "[L "]" \\ L \rightarrow "[L "]" L \\ L \rightarrow "[L "]" \\ L \rightarrow a L \\ L \rightarrow a \end{array} \right.$$

Answer the following questions:

- (a) For the translation function τ_1 that flattens the nested lists, for instance:

$$\tau_1([a[a a][[a]]) = [a a a a]$$

write a BNF translation grammar or scheme G_1^τ (with source grammar G).

- (b) Draw the translation tree for the sample translation below:

$$\tau_1([a[a a][[a]]) = [a a a a]$$

- (c) For the translation function τ_2 that removes all the square brackets and terminates each (sub)list with a single dot ".", for instance:

$$\tau_2([a[a a][[a]]) = a a a . a \dots$$

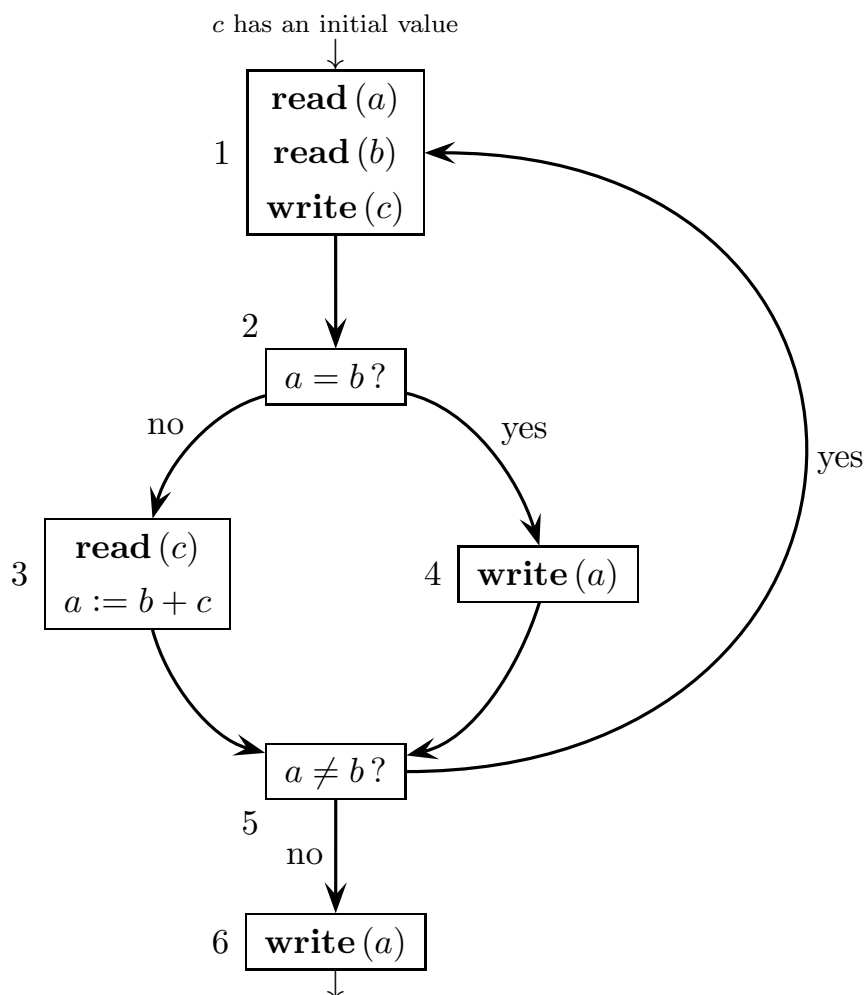
write a BNF translation grammar or scheme G_2^τ (with source grammar G).

- (d) Draw the translation tree for the sample translation below:

$$\tau_2([a[a a][[a]]) = a a a . a \dots$$

- (e) Are the two translations τ_1 and τ_2 invertible? Why?

2. Consider the control flow-graph (CFG) of the program below, with three numerical variables a , b and c (variable c has an initial value passed as input argument), and six nodes numbered 1, $\dots$, 6:

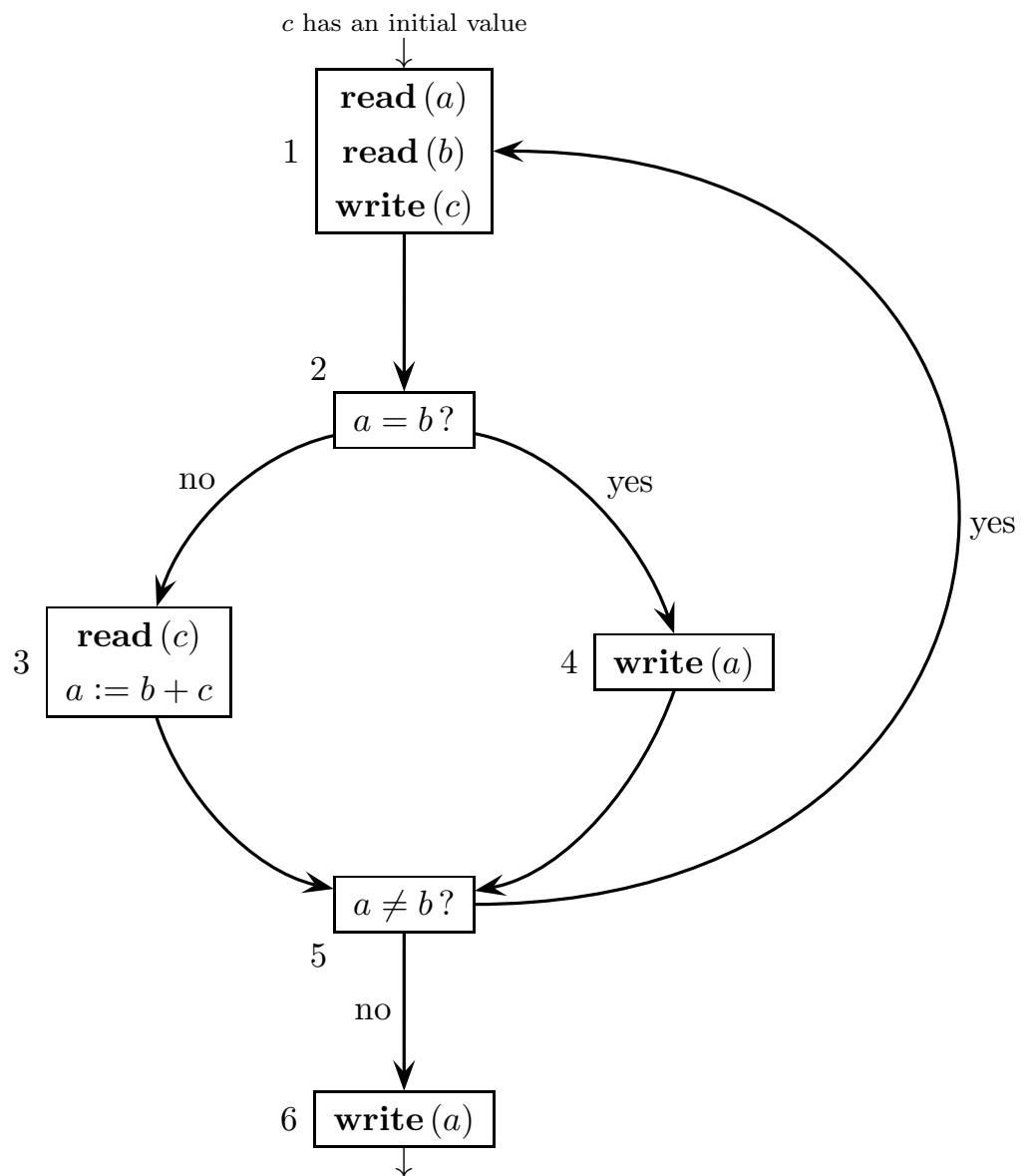


Answer the following questions (use the tables / trees / spaces on the next pages):

- Write a regular expression R that generates the language of the program execution traces, over the alphabet $\{1, \dots, 6\}$, disregarding the program semantics.
- Informally find the live variables and decorate each node of the graph with the variables that are live at the input of the node (see the next pages).
- Write the liveness flow equations for the node 3 and then verify that the live variables informally found at point (b) satisfy such equations.
- How many definitions does the variable a have? Decorate each node of the graph with the definitions of a that reach the output of the node (see the next pages).
- (optional) Now consider the semantics of the program. Can you change the regular expression R to a more restrictive one R' , such that $L(R')$ is strictly contained in $L(R)$?

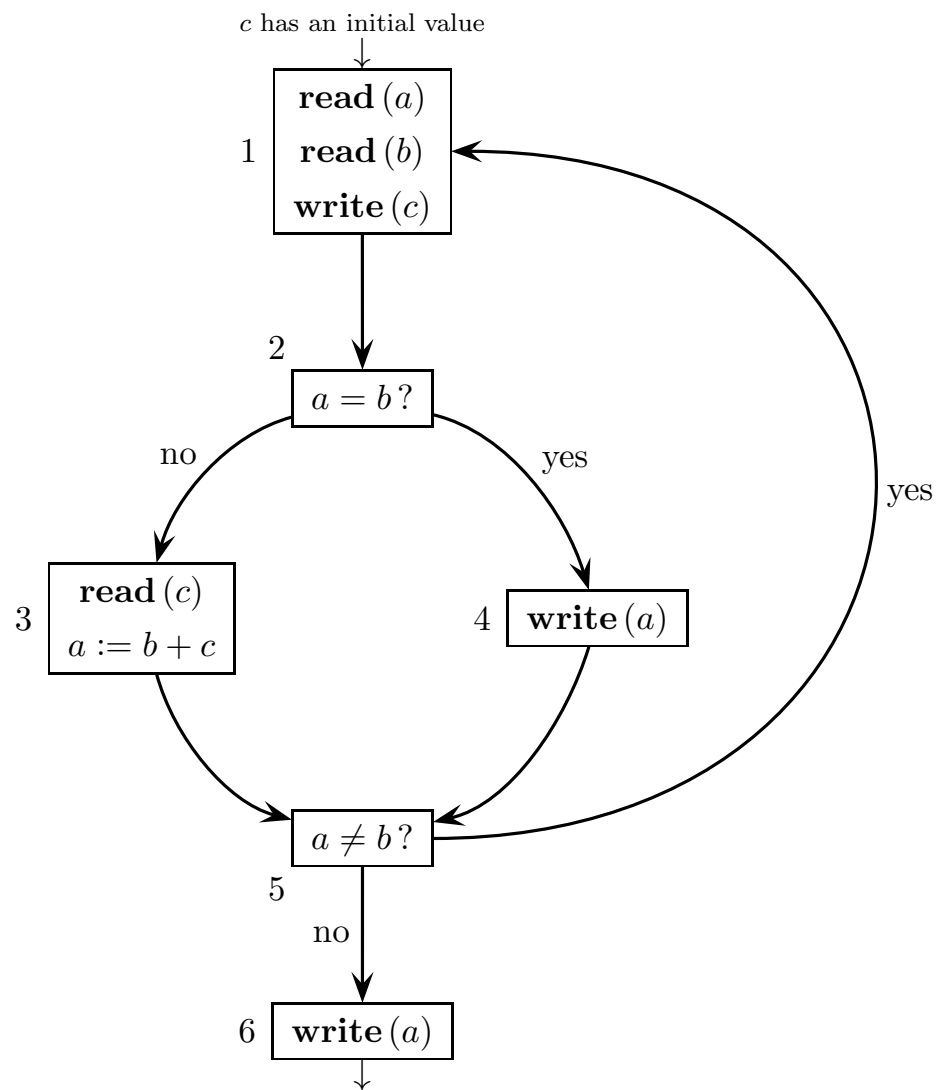
regular expression R – question (a)

live variables (on the node input) – question (b)



liveness equations for node 3 and their verification – question (c)

reaching definitions (on the node output) – question (d)



semantics – question (e)

free space – any questions

free space – any questions

free space – any questions